

DexVision / Hand2Bot

Vision-based dexterous teleoperation, learning feasibility, comprehensive skill data,
and future language-guided orchestration

CURRENT POSITION: Levels 1–2 complete. Level 3 is active and asks whether learning works on the existing data. Level 4 builds the comprehensive dataset; Level 5 learns and qualifies the full skill set.

This overview is generated from docs/project_overview.md. CURRENT_STATUS.md remains the checkpoint source of truth.

Project identity

DexVision is a vision-based dexterous teleoperation, robot-learning dataset, and reusable manipulation-skill project. A camera tracks a human hand, the motion is retargeted to a Shadow Hand in MuJoCo, demonstrations are recorded and validated, and PyTorch policies learn bounded low-level skills.

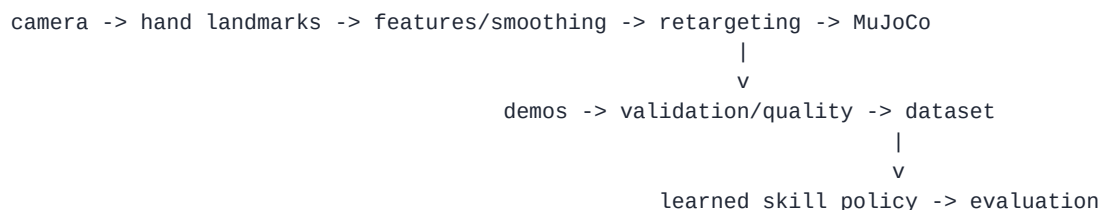
The eventual language model is a planner, not the robot controller. It should turn requests into typed calls to already-qualified skills. A deterministic supervisor validates parameters and world state, enforces safety and timeouts, and reports structured success or failure.

Current status

Levels 1 and 2 are complete. The repository has live full-hand/base/wrist teleoperation, resettable manipulation tasks, demonstration recording and semantic replay, quality filtering and relabeling, dataset summaries, three retargeting methods, a full retargeting benchmark, and an immutable Level 2 dataset release tracked with Git LFS.

Level 3 is active. Its purpose is to test whether the existing narrow datasets support useful behavior-cloned policies. The Level 3.1 goal-conditioned dataset loader is complete; the next checkpoint is Level 3.2, the goal-conditioned MLP policy.

Architecture



future Level 7:

language request -> task plan -> deterministic supervisor -> qualified skill executor

The continuous skill policy receives state plus a typed goal and outputs bounded base/wrist/finger actions. Visual perception produces stable object ids and poses. A language model may help select or disambiguate objects, but it does not emit high-rate actuator commands.

Roadmap

Level 1 — Teleoperation (complete)

OpenCV and MediaPipe track a human hand. Calibrated image motion, hand scale, palm rotation, and finger features control the MuJoCo Shadow Hand through the full Level 1.13 action space.

Level 2 — Demonstrations and data infrastructure (complete)

The system records, replays, validates, relabels, filters, summarizes, and benchmarks demonstrations. The released manipulation data includes 55 `reach_touch_target`, 55 `button_press`, and 101 `push_cube_to_target` clean successes. The archive is immutable and accompanied by a checksum and manifest.

Level 3 — Learning feasibility (active)

Level 3 builds a deterministic PyTorch loader, a small goal-conditioned MLP, a reproducible behavior-cloning loop, and closed-loop MuJoCo evaluation. It then checks whether the same approach works for reach, button press, and cube push, diagnoses the role of data quality and action fields, and adds a temporal baseline only if measured failures justify it.

Passing Level 3 proves the learning pipeline on the Level 2 distributions. It does not establish cross-session, cross-object, visual, or open-world generalization.

Level 4 — Comprehensive skill dataset (planned)

Level 4 is the major data-haul phase. New data must carry genuine session provenance, broader object and goal variation, synchronized visual labels when used, explicit failures, and separately labeled operator corrections. Level 4.0 freezes exact counts after small pilots; the planning envelope is roughly 250–350 new accepted episodes across at least three genuine sessions, three simple rigid-object families, and three or four target regions. Compatible Level 2 episodes remain labeled legacy seed data.

Required operational skills are:

```

reach_object
pick_object
place_held_object
push_object_to_target
press_button

```

`rotate_dial` is optional. Grasp/lift/hold and transport/place/release are measurable phases inside pick and place-held-object, not separate policies. Approximately 100–150 complete pick/place demonstrations can therefore cover those phase labels without multiplying the collection target for every micro-skill.

Level 4 ends with an immutable release, coverage and bias reports, and frozen session/object/goal/visual splits. It does not claim full-scale trained skills.

Visual data starts with simulator truth and rendered boxes, masks, poses, and stable ids. Model training waits for Level 5.

Level 5 — Full-scale skill learning and qualification (planned)

Level 5 retrains the successful Level 3 approach on the comprehensive release and evaluates every core skill on session-held-out and condition-held-out data. It separately measures ground-truth-state and perception-grounded rollouts, tests whether corrective data improve the core policies, and escalates to temporal or action-chunked models only when measured failures justify the complexity. Initial retry and abort behavior remains deterministic rather than a separately learned recovery skill.

Only qualified policies enter the typed skill registry and supervised executor. Level 5 then runs at least three materially different scripted pilot tasks through that same interface.

Visual grounding uses a conventional detector/tracker first. A compact local VLM may be tested for semantic object selection; metric localization and safety remain separate.

Level 6 — Robustness and portfolio polish (planned)

The original portfolio roadmap is preserved here: README and architecture polish, results tables, demo materials, robust CLI/config handling, reproducibility and CI, troubleshooting, optional dataset export, and carefully labeled advanced extensions.

Level 7 — Language-guided orchestration (future)

An LLM or deterministic planner consumes symbolic world state and versioned skill cards. It creates typed plans; a deterministic supervisor validates and executes them. Scripted plans and mock skills validate the contract before learned policies are introduced one at a time.

Tabletop workcell pilot tasks

Level 5 validates three related but materially different scripted pilots through the same typed skill interface:

1. Workspace clearing: return loose rigid parts to bins, using a push for a flat or awkward part when appropriate.
2. Inspection-station operation: place a part on an inspection pad and press Start, with dial setting optional.
3. Workspace setup: arrange components at marked positions for the next job.

A combined final work order can ask the system to place a blue cylinder on the inspection pad, put a red block in the left tray slot, optionally set the dial to 45 degrees, and press Start. General arbitrary-object grasping, learned regrasp/drop recovery, hinged lids, tools, kitchen work, cutting, pouring, liquids, deformables, and open-world scenes are deferred beyond the first complete project.

Dataset release policy

Editable operator data stays under ignored [data/demos/](#). Immutable bounded releases use Git LFS with a manifest and SHA-256 checksum. The Level 2 archive must never be overwritten. If Level 4 images exceed repository hosting quotas, Git should still track manifests, splits, checksums, and retrieval instructions while a versioned external artifact store holds the immutable payload.

Honest project claim

DexVision currently demonstrates a complete teleoperation and dataset engine. Level 3 will determine whether its existing data can support autonomous policies. Level 4 will determine whether a comprehensive dataset can be built; Level 5 will determine whether those data can become a compact, qualified workcell skill library. Language-guided multi-skill behavior remains future work until those evidence gates pass.